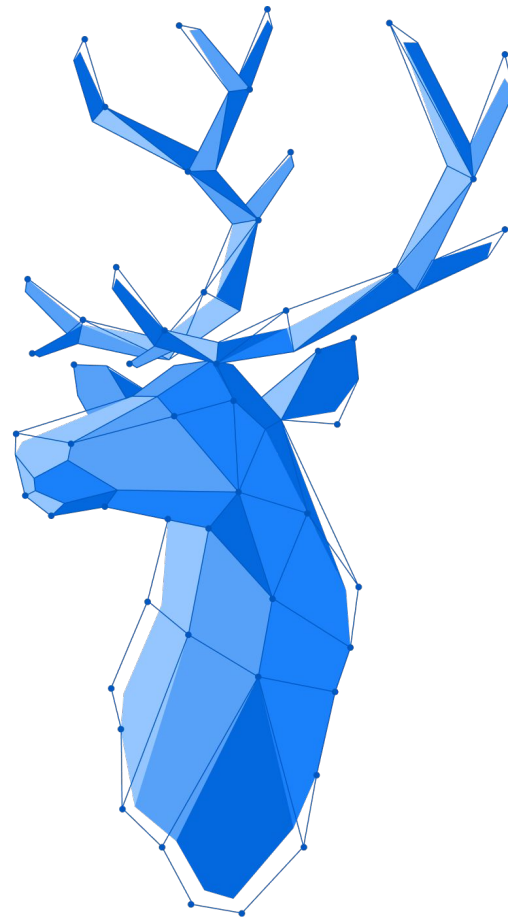




Layer Isolation

By Gerald Koch



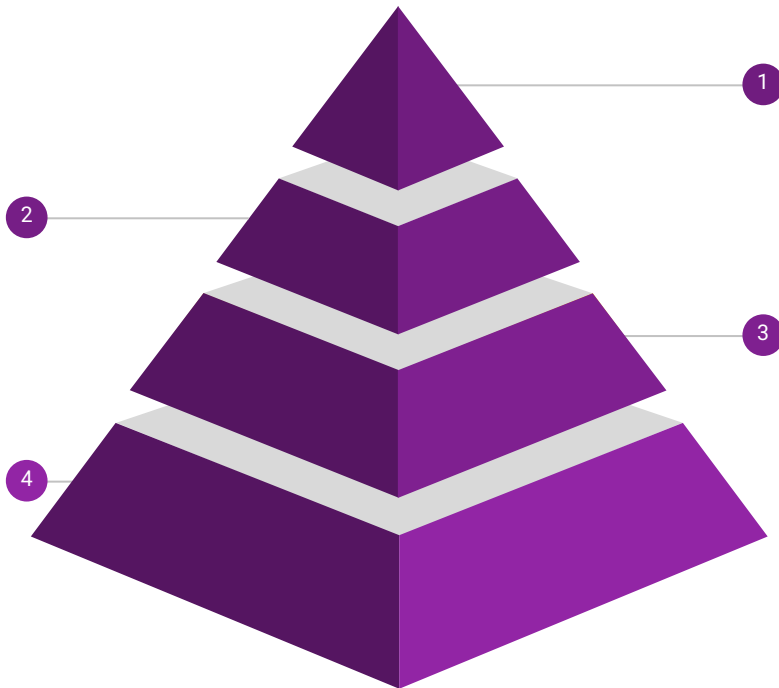
Layers in Architecture

Presenter

- Knows and Uses services to get the data (model)
- Conducts the view
- Knows view over interface
- Has the scope of the view

Persistence

- Layer for data management
- Can connect to database, JSON, XML or other sources
- implements an Interface for easy exchangeability



View

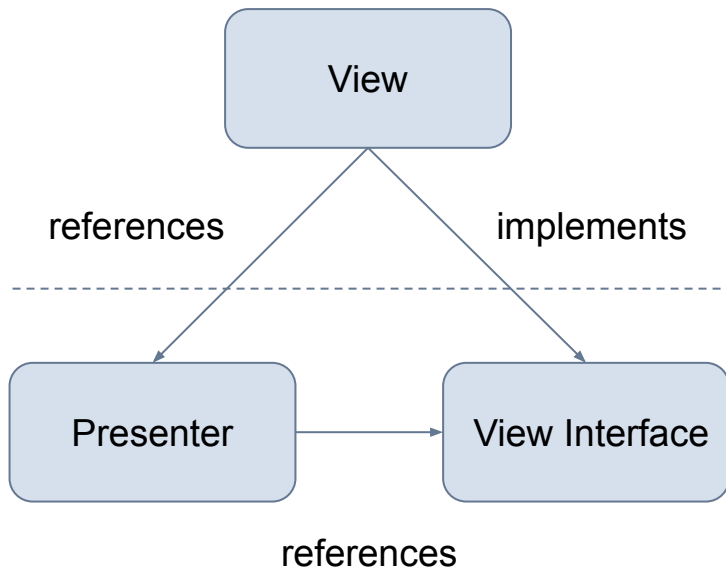
- Shows what the presenter commands
- Dependency to presenter
- Uses only presenter (and given models)

Service

- Knows the persistence via interfaces
- Gets data from persistence and processes them for usage in the presenter/view
- Has the scope Singleton

Layer Isolation

The MVP Pattern



View:

- Implements View Interface
- Can use UI framework classes
- References Presenter
- calls Presenter on input events
- Follows commands of Presenter

View Interface:

- Contains presenter relevant UI methods
- Is referenced from presenter
- Has **NO** UI framework classes

Presenter:

- Conducts the UI
- **NO** UI framework classes
- References the view interface

Layer Isolation

The MVP Pattern – Example Part 1

View Interface

```
public interface ViewInterface {  
    void showTableData(List<SomeObject> tableData);  
}
```

Presenter

```
@Scope(value =  
        ConfigurableBeanFactory.SCOPE_PROTOTYPE)  
@Component  
public class Presenter {  
  
    private ViewInterface view;  
    private final SomeService service;  
  
    public Presenter (SomeService service) {  
        this.service = service;  
    }  
}
```

```
public void init(ViewInterface view) {  
    this.view = view;  
}  
  
public void buttonClicked() {  
    view.showTableData(service.getSomeBeans());  
}  
}
```

Layer Isolation

The MVP Pattern – Example Part 2

View (Vaadin)

```
@UIScope
@Route("")
public class View extends FlexLayout
    implements ViewInterface {

    private final Presenter presenter;
    private final Grid<SomeObject> grid
        = new Grid<>(SomeObject.class);

    //inject the presenter (Spring)
    public View (Presenter presenter) {
        this.presenter = presenter;
        Button button = new Button("Click",
            e -> presenter.buttonClicked());
        add(button, grid);
    }
}
```

```
@Override
public void onAttach(AttachEvent attachEvent) {
    if(attachEvent.isInitialAttach()) {
        presenter.init(this);
    }
}

@Override
public void showTableData(
    List<SomeObject> tableData) {
    grid.setItems(tableData);
}
}
```

Layer Isolation

Software layers as modules

